



---

# **Cadence Driver Verification Plan and Test Report**

**Product Version 1.6.0**

**September 2017**

© 1996-2017 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

**Trademarks:** Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

**Restricted Permission:** This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.
2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

**Disclaimer:** INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

**Restricted Rights:** Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

---

# Cadence Driver Verification Plan and Test Report

Cadence Design Systems

# Table of Contents

|                                                 |   |
|-------------------------------------------------|---|
| 1. Overview .....                               | 1 |
| 1.1. Document Purpose .....                     | 1 |
| 1.2. Acronyms .....                             | 1 |
| 2. Test Equipment/Environment .....             | 2 |
| 2.1. Overview .....                             | 2 |
| 2.2. Driver Tests .....                         | 2 |
| 2.3. Functional Tests .....                     | 2 |
| 3. Test Results .....                           | 3 |
| 3.1. Test Summary .....                         | 3 |
| 3.2. Test Details .....                         | 3 |
| 3.2.1. ASF Functional tests .....               | 3 |
| 3.2.2. nullptr .....                            | 3 |
| 3.2.3. ranges .....                             | 4 |
| 4. Parasoft DTP Engine for C/C++ Analysis ..... | 5 |

# Chapter 1. Overview

## 1.1. Document Purpose

The purpose of this document is to define the API testing for the Cadence driver firmware and to present the results.

## 1.2. Acronyms

|             |                                                                      |
|-------------|----------------------------------------------------------------------|
| API         | Application Programming Interface.                                   |
| Core Driver | Cadence Firmware component that provides IP programming abstraction. |
| SoC         | System on a Chip                                                     |

## Chapter 2. Test Equipment/Environment

### 2.1. Overview

All testing is on a hardware simulated platform, running either a bare-metal or Linux build. A sequence of tests is automatically generated as an executable program during the build process, and run from within the Cadence firmware verification platform environment. Results are logged to a file via Cadence firmware verification platform console output then parsed using a python script and formatted into this report.

### 2.2. Driver Tests

These tests confirm the operation of the core driver API under a range of different inputs, ensuring that operation complies with that described in the user documentation and that there are no unexpected errors. Some tests may not be appropriate for some APIs, depending on configuration, if so these will be marked as "NOT SUPPORTED". For each test which runs, the result will be displayed as "PASSED" or "FAILED". Tests include:

|                                   |                                                                                                                             |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Null pointer tests                | API is checked to confirm that all null pointers are detected, and return appropriate error code                            |
| Get/set and register access tests | API is checked for all get/set or register access functions, and confirm that these can correctly write and read the value. |
| Ranges tests                      | API is checked with valid and invalid parameters, and return code compared with that expected.                              |

NOTE: Not all configurations will require all types of test.

### 2.3. Functional Tests

A sequence of predefined tests is run, with an expected result. These tests exercise the Cadence IP core and API function, and may require use of additional test IP in the Cadence firmware verification platform

## Chapter 3. Test Results

### 3.1. Test Summary

ASF Functional tests: 17 calls, 17 passed

nullptr: 29 calls, 29 passed

ranges: 29 calls, 27 passed, 2 not supported

### 3.2. Test Details

#### 3.2.1. ASF Functional tests

```
ASF_testBIST1 PASSED
ASF_testBIST2 PASSED
ASF_testDisableAllEventsFunc PASSED
ASF_testEnableAllEventsFunc PASSED
ASF_testEnableEventFunc PASSED
ASF_testDisableEventFunc PASSED
ASF_testSetEventAsNonFatalFunc PASSED
ASF_testEnableProtocolEventByMaskFunc PASSED
ASF_testEnableProtocolEventByMaskFunc PASSED
ASF_testDisableProtocolEventByMaskFunc PASSED
ASF_testEnableProtocolEventByIDFunc PASSED
ASF_testDisableProtocolEventByIDFunc PASSED
ASF_testEnableTimeoutEventByMaskFunc PASSED
ASF_testDisableTimeoutEventByMaskFunc PASSED
ASF_testEnableTimeoutEventByIDFunc PASSED
ASF_testDisableTimeoutEventByIDFunc PASSED
ASF_testStatisticFunc PASSED
```

#### 3.2.2. nullptr

```
probe PASSED
init PASSED
destroy PASSED
start PASSED
stop PASSED
isr PASSED
checkIfASFSupported PASSED
getSupportedASF PASSED
getSupportedTimeoutErrors PASSED
getSupportedProtocolErrors PASSED
selfTest PASSED
testEvent PASSED
enableEvent PASSED
disableEvent PASSED
enableAllEvents PASSED
disableAllEvents PASSED
setEventAsNonFatal PASSED
setEventAsFatal PASSED
enableProtocolEventByMask PASSED
enableProtocolEventByID PASSED
disableProtocolEventByMask PASSED
disableProtocolEventByID PASSED
enableTimeoutEventByMask PASSED
enableTimeoutEventByID PASSED
```

disableTimeoutEventByMask PASSED  
disableTimeoutEventByID PASSED  
getStatistic PASSED  
clearStatistic PASSED  
restoreStatistic PASSED

### 3.2.3. ranges

probe PASSED  
init PASSED  
destroy PASSED  
start PASSED  
stop PASSED  
isr PASSED  
checkIfASFSupported PASSED  
getSupportedASF PASSED  
getSupportedTimeoutErrors PASSED  
getSupportedProtocolErrors PASSED  
selfTest PASSED  
testEvent PASSED  
enableEvent PASSED  
disableEvent PASSED  
enableAllEvents PASSED  
disableAllEvents PASSED  
setEventAsNonFatal PASSED  
setEventAsFatal PASSED  
enableProtocolEventByMask NOT SUPPORTED  
enableProtocolEventByID PASSED  
disableProtocolEventByMask NOT SUPPORTED  
disableProtocolEventByID PASSED  
enableTimeoutEventByMask PASSED  
enableTimeoutEventByID PASSED  
disableTimeoutEventByMask PASSED  
disableTimeoutEventByID PASSED  
getStatistic PASSED  
clearStatistic PASSED  
restoreStatistic PASSED



## Chapter 4. Parsoft DTP Engine for C/C++ Analysis

### Summary:

Parsoft DTP Engine for C/C++ Analysis: 53 rules detected  
0 violations.

### Checked rules:

- All exit paths from a function with non-void return type shall have an explicit return statement with an expression
- All member variables should be initialized in constructor
- Always catch exceptions
- Always provide empty brackets ([]) for delete when deallocating arrays
- Avoid accessing arrays out of bounds
- Avoid buffer overflow due to defining incorrect format limits
- Avoid calling virtual functions from constructors
- Avoid conditions that always evaluate to the same value
- Avoid division by zero
- Avoid double locking
- Avoid function duplication
- Avoid infinite loops
- Avoid null pointer dereferencing
- Avoid overflow due to reading a not zero terminated string
- Avoid overflow when reading from a buffer
- Avoid overflow when writing to a buffer
- Avoid race conditions when using fork and file descriptors
- Avoid race conditions while checking for the existence of a symbolic link
- Avoid use before initialization
- Boolean condition always evaluates to the same value due to enumeration with only zero or only non-zero constants
- Class cannot inherit other class more than once unless it is virtual inheritance
- Declare a copy assignment operator for classes with dynamically allocated memory
- Declare a copy constructor for classes with dynamically allocated memory
- Do not abandon unreleased locks
- Do not acquire locks in different order
- Do not call delete on non-pointers
- Do not cast from or to incomplete class at the point of casting
- Do not check for null after dereferencing
- Do not delete objects with incomplete class at the point of deletion
- Do not free resources using invalid pointers
- Do not invoke malloc/realloc for objects having constructors
- Do not pass negative values to functions expecting non-negative arguments
- Do not throw from within destructor
- Do not use blocking functions while holding a lock
- Do not use 'delete' on pointers to a void type
- Do not use resources that have been freed
- Do not use string literals as operands of equality or relational operators
- Ensure resources are freed
- Floating-point expressions shall not be tested for equality or inequality
- If a class has virtual functions it shall have a virtual destructor
- Local variables should not use the same names as member variables
- McCabe Cyclomatic Complexity
- Nested Blocks Depth
- Never provide brackets ([]) for delete when deallocating non-arrays
- Pass objects by reference instead of by value
- Pointer arithmetic performed on freshly allocated memory
- Pointer should not be compared with NULL using relational operators  
&lt;, >, >=, &lt;=
- Properly terminate character strings
- Restore stream format
- Suspicious argument to malloc
- Suspicious setting of stream flags

The address of an object with automatic storage shall not be returned from a  
function  
Throw by value, catch by reference